Variable selection: an introduction

Lino Galiana
Version of November 2, 2026

Access to ever-richer databases enables increasingly refined modeling. However, parsimonious models are generally preferable to extremely rich models for obtaining good performance on a new dataset (*out-of-sample* predictions). Variable selection methods, such as `LASSO`, can be used to select the most relevant signal diluted amidst the noise when there is a lot of information to process.

⚠️ **Automatically generated PDF.** This document is produced from the course website and may contain formatting issues. For an up-to-date version adapted to your reading, visit the course page: pythonds.linogaliana.fr.

Table of contents

1 The Principle of LASSO **2**
1.1 General Principle 2
1.2 Regularization 2
1.3 First LASSO Regression 3
2 Role of the Penalty α in Variable Selection **7**
3 Cross-Validation to Select the Model **7**
Informations additionnelles 9

Machine learning materials in this course uses a unique dataset, presented in the introduction. All examples are based on US county level presidential election results combined with sociodemographic variables. Source code for data ingestion is available on `Github`.

```
!pip install geopandas openpyxl plotnine plotly
```

```
import requests

url     =     'https://raw.githubusercontent.com/linogaliana/python-datascientist/main/content/
modelisation/get_data.py'
r = requests.get(url, allow_redirects=True)
open('getdata.py', 'wb').write(r.content)

import getdata
votes = getdata.create_votes_dataframes()
```

So far, we have assumed that the variables useful for predicting the Republican vote were known to the modeler. Thus, we have only used a limited portion of the available variables in our data. However, beyond the computational burden of building a model with a large number of variables, choosing a limited number of variables (a parsimonious model) reduces the risk of overfitting.

How, then, can we choose the right number of variables and the best combination of these variables? There are multiple methods, including:

- Relying on statistical performance criteria that penalize non-parsimonious models. For example, BIC.
- *Backward elimination* techniques.

- Building models where the statistic of interest penalizes the lack of parsimony (which is what we aim to do here).

In this chapter, we will present the main challenges of variable selection through LASSO.

We will subsequently use the following functions or packages:

```
import numpy as np
from sklearn.svm import LinearSVC
from sklearn.feature_selection import SelectFromModel
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Lasso
import sklearn.metrics
from sklearn.linear_model import LinearRegression
import matplotlib.cm as cm
import matplotlib.pyplot as plt
from sklearn.linear_model import lasso_path
import seaborn as sns
```

1 The Principle of LASSO

1.1 General Principle

The class of *feature selection* models is very broad and includes a diverse range of models. We will focus on LASSO (*Least Absolute Shrinkage and Selection Operator*), which is an extension of linear regression aimed at selecting *sparse* models. This type of model is central to the field of *Compressed sensing* (where the term *L1-regularization* is more commonly used than LASSO). LASSO is a special case of elastic-net regressions, with another famous case being *ridge regression*. Unlike classical linear regression, these methods also work in a framework where $p > N$, i.e., where the number of predictors is much larger than the number of observations.

1.2 Regularization

By adopting the principle of a penalized objective function, LASSO allows certain coefficients to be set to 0. Variables with non-zero norms thus pass the selection test.

LASSO optimization program

LASSO is a constrained optimization problem. It seeks to find the estimator β that minimizes the quadratic error (linear regression) under an additional constraint regularizing the parameters: $$ \min_{\beta} \frac{1}{2}\mathbb{E}\bigg(\big(X\beta - y \big)^2 \bigg) \\ \text{s.t. } \sum_{j=1}^p |\beta_j| \leq t $$

This program is reformulated using the Lagrangian, allowing for a more tractable minimization program:

$$\beta^{\text{LASSO}} = \arg\min_{\beta} \frac{1}{2}\mathbb{E}\bigg(\big(X\beta - y\big)^2 \bigg) + \alpha \sum_{j=1}^{p} |\beta_j| = \arg\min_{\beta} ||y - X\beta||_2^2 + \lambda ||\beta||_1$$

where λ is a reformulation of the previous regularization term, depending on α. The strength of the penalty applied to non-parsimonious models depends on this parameter.

1.3 First LASSO Regression

As we aim to find the best predictors of the Republican vote, we will remove variables that can be directly derived from these: the competitors' scores!

```
import pandas as pd

df2 = pd.DataFrame(votes.drop(columns='geometry'))
df2 = df2.loc[
  :,
  ~df2.columns.str.endswith(
    ('_democrat','_green','_other', 'winner', 'per_point_diff', 'per_dem')
    )
  ]


df2 = df2.loc[:,~df2.columns.duplicated()]
```

We will use the following function to represent correlation matrix.

```
import numpy as np
import pandas as pd
import plotly.express as px


def plot_corr_heatmap(
    df: pd.DataFrame,
    drop_cols=None,
    column_labels: dict | None = None,
    decimals: int = 2,
    width: int = 600,
    height: int = 600,
    show_xlabels: bool = False
):
    """
    Trace une heatmap de corrélation (triangle inférieur) à partir d'un DataFrame.

    Paramètres
    ----------
    df : pd.DataFrame
        DataFrame d'entrée.
    drop_cols : list ou None
        Liste de colonnes à supprimer avant le calcul de la corrélation
        (ex: ['winner']).
    column_labels : dict ou None
        Dictionnaire pour renommer les colonnes (ex: column_labels).
    decimals : int
        Nombre de décimales pour l'arrondi avant corr().
    width, height : int
        Dimensions de la figure Plotly.
    show_xlabels : bool
        Afficher ou non les labels en abscisse.
    """
```

```
    data = df.copy()

    # 1. Colonnes à drop
    if drop_cols is not None:
        data = data.drop(columns=drop_cols)

    # 2. Arrondi + renommage éventuel
    if column_labels is not None:
        data = data.rename(columns=column_labels)
    data = data.round(decimals)

    # 3. Matrice de corrélation
    corr = data.corr()

    # 4. Masque triangle supérieur
    mask = np.triu(np.ones_like(corr, dtype=bool))
    corr_masked = corr.mask(mask)

    # 5. Heatmap Plotly
    fig = px.imshow(
        corr_masked.values,
        x=corr.columns,
        y=corr.columns,
        color_continuous_scale='RdBu_r',  # échelle inversée
        zmin=-1,
        zmax=1,
        text_auto=".2f"
    )

    # 6. Hover custom
    fig.update_traces(
        hovertemplate="Var 1: %{y}<br>Var 2: %{x}<br>Corr: %{z:.2f}<extra></extra>"
    )

    # 7. Layout
    fig.update_layout(
        coloraxis_showscale=False,
        xaxis=dict(
            showticklabels=show_xlabels,
            title=None,
            ticks=''
        ),
        yaxis=dict(
            showticklabels=show_xlabels,
            title=None,
            ticks=''
        ),
        plot_bgcolor="rgba(0,0,0,0)",
        margin=dict(t=10, b=10, l=10, r=10),
        width=width,
        height=height
    )

    return fig
```

In this exercise, we will also use a function to extract the variables selected by LASSO, here it is:

```
from sklearn.linear_model import Lasso
from sklearn.pipeline import Pipeline

def extract_features_selected(lasso: Pipeline, preprocessing_step_name: str = 'preprocess') -
> pd.Series:
    """
    Extracts selected features based on the coefficients obtained from Lasso regression.

    Parameters:
    - lasso (Pipeline): The scikit-learn pipeline containing a trained Lasso regression model.
    - preprocessing_step_name (str): The name of the preprocessing step in the pipeline. Default
is 'preprocess'.

    Returns:
    - pd.Series: A Pandas Series containing selected features with non-zero coefficients.
    """
    # Check if lasso object is provided
    if not isinstance(lasso, Pipeline):
        raise ValueError("The provided lasso object is not a scikit-learn pipeline.")

    # Extract the final transformer from the pipeline
    lasso_model = lasso[-1]

    # Check if lasso_model is a Lasso regression model
    if not isinstance(lasso_model, Lasso):
        raise ValueError("The final step of the pipeline is not a Lasso regression model.")

    # Check if lasso model has 'coef_' attribute
    if not hasattr(lasso_model, 'coef_'):
       raise ValueError("The provided Lasso regression model does not have 'coef_' attribute. "
                        "Make sure it is a trained Lasso regression model.")

    # Get feature names from the preprocessing step
    features_preprocessing = lasso[preprocessing_step_name].get_feature_names_out()

    # Extract selected features based on non-zero coefficients
    features_selec = pd.Series(features_preprocessing[np.abs(lasso_model.coef_) > 0])

    return features_selec
```

Exercise 1: First LASSO

We are still trying to predict the variable `per_gop`. Before making our estimation, we will create certain intermediate objects to define our *pipeline*:

1. In our `DataFrame`, replace infinite values with `NaN`.
2. Create a training sample and a test sample.

Now we can move on to defining our *pipeline*. This example might serve as inspiration, as well as this one.

3. First, create the *preprocessing* steps for our model. For this, it is common to separate the steps applied to continuous numerical variables from those applied to categorical variables.

- For **numerical variables**, impute with the mean and then standardize;
- For **categorical variables**, linear regression techniques require using one-hot encoding. Before performing one-hot encoding, impute with the most frequent value.

4. Finalize the *pipeline* by adding the estimation step and then estimate a LASSO model penalized with $\alpha = 0.1$.

Assuming your *pipeline* is stored in an object named `pipeline` and the last step is named `model`, you can directly access this step using the object `pipeline['model']`.

5. Display the coefficient values. Which variables have a non-zero value?
6. Show that the selected variables are sometimes highly correlated.
7. Compare the performance of this parsimonious model with that of a model with more variables.

Help for Question 1

```
# Replace infinities with NaN
df2.replace([np.inf, -np.inf], np.nan, inplace=True)
```

Help for Question 3 The pipeline definition follows this structure:

```
numeric_pipeline = Pipeline(steps=[
    ('impute', # define the imputation method here
     ),
    ('scale', # define the standardization method here
    )
])

categorical_pipeline = # adapt the template

# Define numerical_features and categorical_features beforehand
preprocessor = ColumnTransformer(transformers=[
    ('number', numeric_pipeline, numerical_features),
    ('category', categorical_pipeline, categorical_features)
])
```

The *preprocessing* pipeline (question 3) takes the following form:

The *pipeline* takes the following form once finalized (question 4):

At the end of question 5, the selected variables are:

The model is quite parsimonious as it uses a subset of our initial variables (especially since our categorical variables have been split into numerous variables through *one hot encoding*).

Some variables make sense, such as education-related variables. Notably, one of the best predictors for the Republican score in 2020 is… the Republican score (and mechanically the Democratic score) in 2016 and 2012.

Additionally, redundant variables are being selected. A more thorough data cleaning phase would actually be necessary.

The parsimonious model is (slightly) more performant:

Moreover, it can already be noted that regressing the 2020 score on the 2016 score results in very good explanatory performance, suggesting that voting behaves like an autoregressive process:

```
import statsmodels.api as sm
import statsmodels.formula.api as smf
smf.ols("per_gop ~ share_2016_REPUBLICAN", data = df2).fit().summary()
```

2 Role of the Penalty α in Variable Selection

So far, we have taken the hyperparameter α as given. What role does it play in the conclusions of our modeling? To investigate this, we can explore the effect of its value on the number of variables passing the selection step.

For the next exercise, we will consider exclusively quantitative variables to speed up the computations. Indeed, with non-parsimonious models, the multiple categories of our categorical variables make the optimization problem difficult to handle.

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

df2.replace([np.inf, -np.inf], np.nan, inplace=True)
X_train, X_test, y_train, y_test = train_test_split(
    df2.drop(["per_gop"], axis = 1),
    100*df2['per_gop'], test_size=0.2, random_state=0
)

numerical_features = X_train.select_dtypes(include='number').columns.tolist()
categorical_features = X_train.select_dtypes(exclude='number').columns.tolist()

numeric_pipeline = Pipeline(steps=[
    ('impute', SimpleImputer(strategy='mean')),
    ('scale', StandardScaler())
])

preprocessed_features = pd.DataFrame(
  numeric_pipeline.fit_transform(
    X_train.drop(columns = categorical_features)
  )
)

numeric_pipeline
```

Exercise 2: Role of the Penalty Parameter

Use the `lasso_path` function to evaluate the number of parameters selected by LASSO as α varies (explore $\alpha \in [0.001, 0.01, 0.02, 0.025, 0.05, 0.1, 0.25, 0.5, 0.8, 1.0]$).

The relationship you should obtain between α and the number of parameters is as follows:

We see that the higher α is, the fewer variables the model selects.

3 Cross-Validation to Select the Model

Which α should be preferred? For this, cross-validation should be performed to choose the model for which the variables passing the selection phase best predict the Republican outcome.

```
from sklearn.linear_model import LassoCV

my_alphas = np.array([0.001,0.01,0.02,0.025,0.05,0.1,0.25,0.5,0.8,1.0])

lcv = (
  LassoCV(
    alphas=my_alphas,
    fit_intercept=False,
    random_state=0,
    cv=5
    ).fit(
      preprocessed_features, y_train
    )
)
```

The *"best"* α can be retrieved as follows:

```
print("alpha optimal :", lcv.alpha_)
```

This can be used to run a new *pipeline*:

```
from sklearn.compose import make_column_transformer, ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_pipeline = Pipeline(steps=[
    ('impute', SimpleImputer(strategy='mean')),
    ('scale', StandardScaler())
])

categorical_pipeline = Pipeline(steps=[
    ('impute', SimpleImputer(strategy='most_frequent')),
    ('one-hot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor = ColumnTransformer(transformers=[
    ('number', numeric_pipeline, numerical_features),
    ('category', categorical_pipeline, categorical_features)
])

model = Lasso(
  fit_intercept=False,
  alpha = lcv.alpha_
)

lasso_pipeline = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('model', model)
])

lasso_optimal = lasso_pipeline.fit(X_train,y_train)

features_selec2 = extract_features_selected(lasso_optimal)
```

Selected features are:

Tip

If the model appears to be insufficiently parsimonious, it would be necessary to revisit the variable definition phase to determine whether different scales for some variables might be more appropriate (e.g., using the `log`).

Informations additionnelles

i Python environment

This site was built automatically through a `Github` action using the `Quarto` reproducible publishing software (version 1.10.18).

The environment used to obtain the results is reproducible via uv. The `pyproject.toml` file used to build this environment is available on the `linogaliana/python-datascientist` repository

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "⩾3.13,<3.14"
dependencies = [
    "altair⩾6.0.0",
    "cartiflette⩾0.2",
    "contextily==1.6.2",
    "duckdb⩾0.10.1",
    "folium⩾0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables⩾0.12.0",
    "gt-extras⩾0.0.8",
    "ipykernel⩾6.29.5",
    "jupyter⩾1.1.1",
    "jupyter-cache⩾1.0.0",
    "kaleido⩾0.2.1",
    "langchain-community⩾0.3.27",
    "loguru==0.7.3",
    "markdown⩾3.8",
    "nbclient⩾0.10.0",
    "nbformat⩾5.10.4",
    "nltk⩾3.9.1",
    "openpyxl⩾3.1.5",
    "pandas⩾3.0",
    "pip⩾25.1.1",
    "plotly⩾6.1.2",
    "plotnine⩾0.15",
    "polars⩾1.8.2",
    "py7zr⩾1.1.3",
    "pyarrow⩾17.0.0",
    "pynsee⩾0.1.8",
    "python-dotenv⩾1.0.1",
    "python-frontmatter⩾1.1.0",
    "pywaffle⩾1.1.1",
    "requests⩾2.32.3",
    "scikit-image⩾0.24.0",
    "scikit-learn⩾1.8.0",
```

```
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

To use exactly the same environment (version of `Python` and *packages*), please refer to the documentation for `uv`.